
Analisis Penjualan BigMart

Dibuat Oleh:

Asma' Syahidah Faisal

Student Dibimbing Data Analytics Offline Batch 3



ALUR PRESENTASI

Self & Project Overview

Problem Statement

Data Exploration

Data Cleaning & Manipulation

Insights



About me



Asma' Syahidah Faisal

Email: asmasyafaisal11@gmail.com

LinkedIn: [linkedin.com/in/asma-syahidah](https://www.linkedin.com/in/asma-syahidah)

- Dibimbing - Bootcamp in Data Analytics (2026 - Sekarang)
- S1 - Informatika (2021 - 2025)



Project overview

analisis loyalitas karyawan per departemen berdasarkan gaji & status perkawinan

Faktor Loyalitas: Stabilitas hidup dan budaya departemen jauh lebih berpengaruh terhadap loyalitas daripada besaran gaji.

Pilar Stabilitas: Karyawan Menikah/Widowed adalah kelompok paling setia; karyawan Lajang/Bercerai paling berisiko resign.

Gap Departemen: Divisi Production & Sales sangat solid, sementara Software Engineering mengalami krisis retensi yang serius.

Risiko Karier: Tanpa jenjang karier yang jelas, talenta teknis dan karyawan muda cenderung menjadikan perusahaan hanya sebagai "tempat singgah



Problem Statement

Ide dasar:

BigMart memiliki berbagai jenis format toko, mulai dari toko kelontong kecil (Grocery Store) hingga supermarket besar. Manajemen perlu memahami apakah perbedaan format fisik dan model bisnis ini memberikan dampak yang signifikan terhadap omzet.

Hipotesa

membandingkan H_0 yang menyatakan tidak adanya perbedaan performa antar jenis toko melawan H_1 yang menduga adanya perbedaan signifikan, di mana tipe toko yang lebih besar seperti Supermarket Type 3 diprediksi memiliki hasil penjualan yang jauh lebih tinggi dibandingkan tipe lainnya.



DATA EXPLORATION

Sumber Data

Data berbentuk CSV, bersumber dari Kaggle

Pengenalan Data

Data terdiri dari 8523 baris dan 12 kolom

Langkah Import Data

1. Menyimpan dataset ke dalam Jupyter Notebook
2. Lalu panggil dengan query:

```
import pandas as pd

df = pd.read_csv("BigMart Sales Data.csv")
df
```



MISSING VALUES, TYPOS & DUPLICATE

```
df.isnull().sum()
```

```
Item_Identifier      0
Item_Weight         1463
Item_Fat_Content      0
Item_Visibility      0
Item_Type            0
Item_MRP            0
Outlet_Identifier    0
Outlet_Establishment_Year  0
Outlet_Size         2410
Outlet_Location_Type  0
Outlet_Type          0
Item_Outlet_Sales    0
dtype: int64
```

Terdapat missing values di 2 kolom

```
print(df['Item_Fat_Content'].value_counts())
```

```
Low Fat      5089
Regular      2889
LF           316
reg          117
low fat      112
Name: Item_Fat_Content, dtype: int64
```

Ditemukan typo pada kolom Item_Fat_Content

```
df.duplicated().sum()
```

```
0
```

tidak ditemukan duplikat

```
# Mencari data yang Item_Visibility-nya nol
visibilitas_nol = data_copy[data_copy['Item_Visibility'] == 0]

# Tampilkan berapa banyak baris yang nol dan contoh datanya
print(f"Jumlah baris dengan visibilitas 0: {len(visibilitas_nol)}")
display(visibilitas_nol[['Item_Identifier', 'Item_Type', 'Item_Visibility']].head())

Jumlah baris dengan visibilitas 0: 526
```

Terdapat data yang bernilai 0 pada kolom Item_Visibility



DATA CLEANING

Mengatasi missing values

Pada kolom Item_weight

```
# Isi data kosong berdasarkan mean ID Produk
data_copy['Item_Weight'] =
data_copy['Item_Weight'].fillna(data_copy.groupby('Item_I
dentifier')['Item_Weight'].transform('mean'))
print(data_copy[['Item_Identifier',
'Item_Weight']].head(20))
```

Pada kolom Outlet-Size

```
# 1. Fungsi untuk mengambil modus agar
tidak IndexError
def get_mode(x):
    m = x.mode()
    return m.iloc[0] if not m.empty else
np.nan
data_copy.head(15)
```



DATA CLEANING

Mengatasi Typos

Pada kolom Item_Fat_Content, ubah ke format standar

Mengganti semua variasi penulisan ke format standar

```
data_copy['Item_Fat_Content'] =
data_copy['Item_Fat_Content'].replace({
    'LF': 'Low Fat',
    'low fat': 'Low Fat',
    'reg': 'Regular'
})
```

Cek hasil akhir

```
print("Hasil setelah dibersihkan:")
print(data_copy['Item_Fat_Content'].value_counts())
```



DATA CLEANING

Handling Visibilitas 0

```
import numpy as np

# 1. Ubah angka 0 menjadi NaN supaya bisa dideteksi oleh fungsi fillna
data_copy['Item_Visibility'] = data_copy['Item_Visibility'].replace(0, np.nan)

# 2. Imputasi nilai NaN dengan Median berdasarkan Item_Type
data_copy['Item_Visibility'] = data_copy['Item_Visibility'].fillna(
    data_copy.groupby('Item_Type')['Item_Visibility'].transform('median')
)

# 3. Validasi akhir: pastikan sudah tidak ada angka 0
print(f"Jumlah baris dengan visibilitas 0 sekarang: {len(data_copy[data_copy['Item_Visibility'] == 0])}")
```

```
Jumlah baris dengan visibilitas 0 sekarang: 0
```

Mengubahnya menjadi NaN (Not a Number), Untuk memberi tahu Python bahwa data ini 'hilang', sehingga tidak akan merusak perhitungan statistik sampai dilakukan pengisian (imputasi) dengan nilai yang lebih masuk akal



INKONSISTENSI FORMAT & TIPE DATA

Memperbaiki spasi berlebih & format huruf

1. Ambil semua kolom yang tipenya Kategorikal (Object)

```
kolom_kategori = data_copy.select_dtypes(include=['object']).columns
```

2. Loop untuk merapikan teks ke Title Case (Huruf depan kapital)

```
for col in kolom_kategori:
```

```
    data_copy[col] = data_copy[col].str.title()
```

3. Khusus untuk Identifier (ID), kita timpa jadi UPPER (Huruf besar semua)

```
kolom_id = ['Item_Identifier', 'Outlet_Identifier']
```

```
for col in kolom_id:
```

```
    data_copy[col] = data_copy[col].str.upper()
```

```
display(data_copy[kolom_kategori].head(10))
```



OUTLIERS

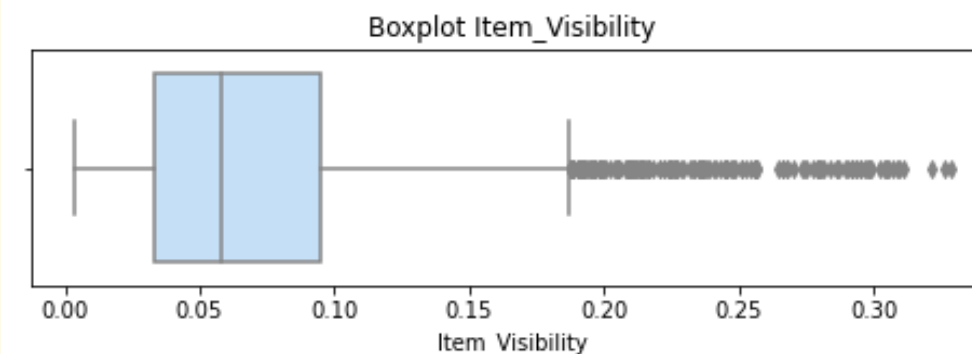
Analisa IQR pada kolom yang bertipe data numerik

```
import seaborn as sns
import matplotlib.pyplot as plt
Q1 = data_copy['Item_Weight'].quantile(0.25)
Q3 = data_copy['Item_Weight'].quantile(0.75)
IQR = Q3 - Q1

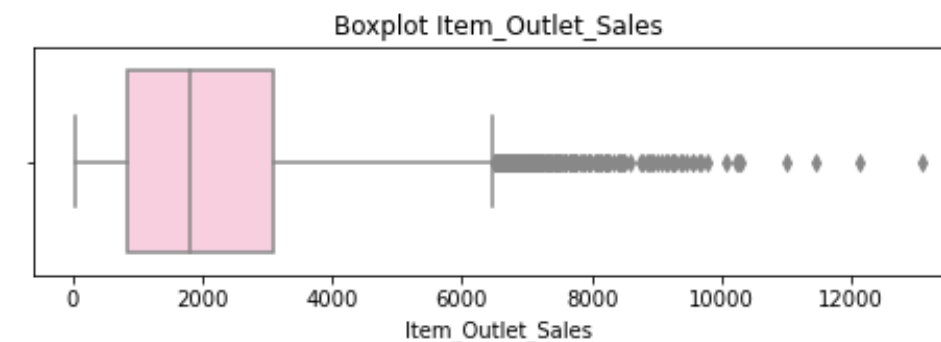
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outlier_weight = data_copy[(data_copy['Item_Weight'] < lower) |
                             (data_copy['Item_Weight'] > upper)]
print(f"Jumlah Outlier Item_Weight: {len(outlier_weight)}")
plt.figure(figsize=(8, 2))
sns.boxplot(x=data_copy['Item_Weight'], color='#ffafcc')
plt.title('Boxplot Item_Weight')
plt.show()
```

Ditemukan 173 baris data outlier.



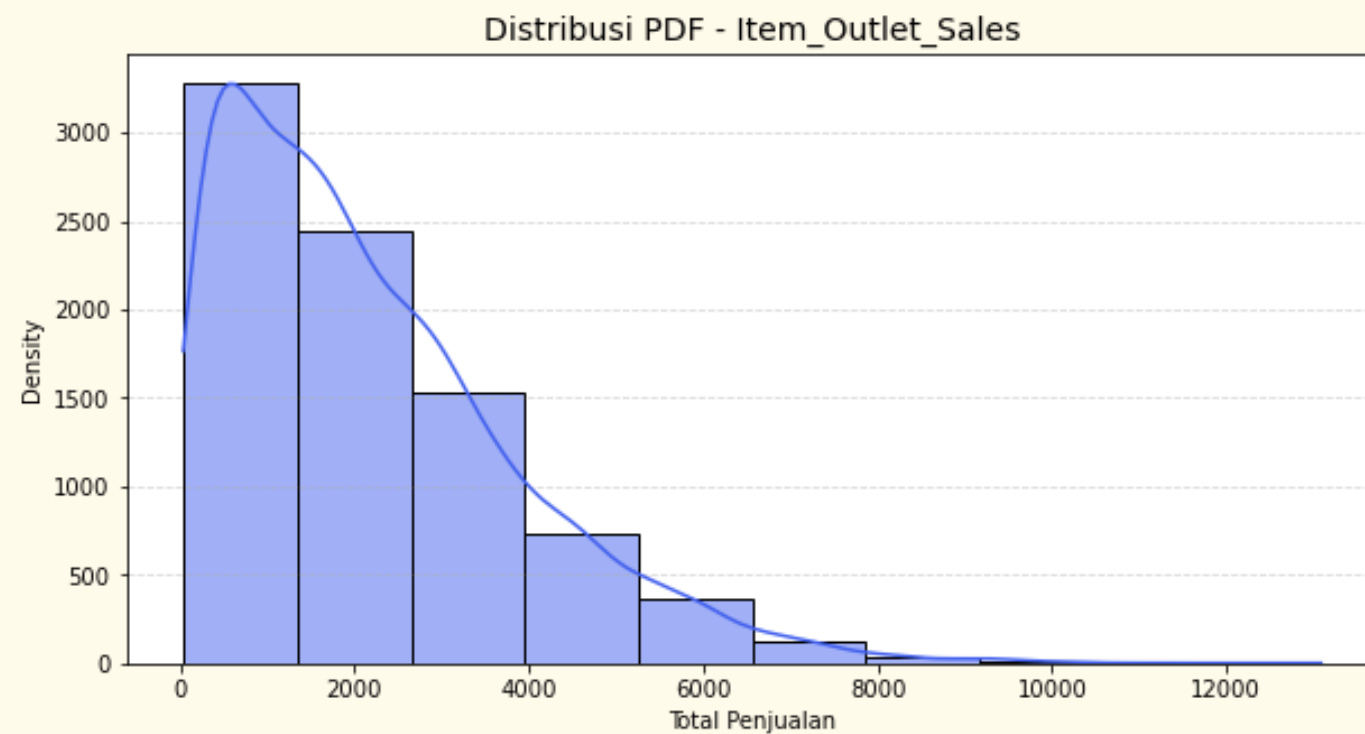
Jumlah Outlier Item_Outlet_Sales: 186



Ditemukan Outliers pada kolom Outlet_Sales & Item_Visibility



DISTRIBUSI



Data ini tidak terdistribusi normal karena bentuknya yang miring ke kanan.

Distribusi ini sangat umum di dunia nyata (terutama retail).

Di mana segelintir outlet memberikan kontribusi penjualan yang sangat besar (high-performing outlets), sementara sisanya beroperasi di level moderat.



DISTRIBUSI

Uji normalisasi dengan KOLMOGOROV-SMIRNOV

```
from scipy.stats import kstest

# Menyiapkan data
data_weight = data_copy['Item_Weight'].dropna()
data_std_weight = (data_weight - data_weight.mean()) / data_weight.std()

# Uji Kolmogorov-Smirnov
stat, p_value = kstest(data_std_weight, 'norm')

print("=== UJI KOLMOGOROV-SMIRNOV: ITEM_WEIGHT ===")
print(f"Statistik K-S: {stat:.4f}")
print(f"P-Value      : {p_value:.4f}")
|
if p_value > 0.05:
    print("Kesimpulan   : Data Berdistribusi NORMAL")
else:
    print("Kesimpulan   : Data TIDAK NORMAL")

=== UJI KOLMOGOROV-SMIRNOV: ITEM_WEIGHT ===
Statistik K-S: 0.0795
P-Value      : 0.0000
Kesimpulan   : Data TIDAK NORMAL
```

- Menggunakan uji Kolmogorov-Smirnov (K-S) untuk membuktikan secara matematis apakah kolom Item_Weight (berat barang) mengikuti distribusi normal atau tidak.



HANDLING OUTLIERS

Menghandling outliers dengan RobustScaler

```
from sklearn.preprocessing import RobustScaler
```

```
# Inisialisasi Scaler yang tahan banting terhadap outliers  
scaler = RobustScaler()
```

```
# Scaling kolom yang ada outliers-nya  
data_copy[['Item_Visibility', 'Item_Outlet_Sales']] =  
scaler.fit_transform(data_copy[['Item_Visibility',  
'Item_Outlet_Sales']])
```

```
print("Scaling Selesai! Outliers sudah ditangani dengan  
RobustScaler.")
```

- Memilih RobustScaler karena dataset BigMart ini memiliki banyak outlier dan distribusinya tidak normal (skewed)
- RobustScaler bekerja menggunakan Median dan IQR, sehingga skala data yang dihasilkan lebih stabil dan objektif. Dengan metode ini, saya bisa melakukan standarisasi data tanpa harus membuang outlier yang sebenarnya masih memiliki nilai informasi bisnis penting bagi analisis saya.



DATA MANIPULATION

Filtering & Sorting berdasarkan Item_Outlet_Sales

1. Pilih ambang batas (Median agar adil 50:50)

```
batas_sales = data_copy['Item_Outlet_Sales'].median()
```

- Melakukan filtering untuk menjamin integritas data penjualan.

2. Filtering Produk yang penjualannya di atas rata-rata (High Performance)

```
df_filter = data_copy[data_copy['Item_Outlet_Sales'] >
batas_sales]
```

- Melakukan sorting untuk melihat data penjualan dari yang terbesar ke terkecil.

3. Cek hasil sederhana

```
print(f"Batas Penjualan: {batas_sales}")
print(f"Jumlah produk 'High Sales': {len(df_filter)}
baris")
df_filter.head()
```



DATA MANIPULATION

Grouping & Feature Engineering

1. Pilih ambang batas (Median agar adil 50:50)

```
batas_sales =  
data_copy['Item_Outlet_Sales'].median()
```

2. Filtering Produk yang penjualannya di atas rata-rata (High Performance)

```
df_filter = data_copy[data_copy['Item_Outlet_Sales'] >  
batas_sales]
```

3. Cek hasil sederhana

```
print(f"Batas Penjualan: {batas_sales}")  
print(f"Jumlah produk 'High Sales': {len(df_filter)}  
baris")  
df_filter.head()
```

- Melakukan Grouping untuk menghitung nilai total dan rata-rata penjualan untuk tiap jenis toko. Dan agar bisa melihat secara adil apakah performa tinggi sebuah toko itu karena tipe tokonya atau cuma kebetulan saja.

- Langkah ini dilakukan untuk menciptakan variabel baru yang merepresentasikan harga produk per satuan unit berat. Variabel ini berfungsi untuk menstandarisasi nilai harga produk sehingga kita dapat menganalisis apakah efisiensi harga per berat memiliki korelasi terhadap total penjualan di berbagai jenis outlet.



DATA MANIPULATION

Encoding

1. Ordinal Encoding (Untuk Kolom Berjenjang)

```
: # Encoding untuk Outlet_Size dan Outlet_Location_Type
# Alasan: Kedua kolom ini memiliki urutan logis (kecil ke besar / tingkat lokasi).

mapping_size = {'Small': 0, 'Medium': 1, 'High': 2}
mapping_location = {'Tier 3': 0, 'Tier 2': 1, 'Tier 1': 2}

df_feature['Outlet_Size_Encoded'] = df_feature['Outlet_Size'].map(mapping_size)
df_feature['Outlet_Location_Type_Encoded'] = df_feature['Outlet_Location_Type'].map(mapping_location)

print("Ordinal Encoding Selesai!")
display(df_feature[['Outlet_Size', 'Outlet_Size_Encoded', 'Outlet_Location_Type', 'Outlet_Location_Type_Encoded']].head())
```

Ordinal Encoding Selesai!

Data ini memiliki urutan yang jelas. 'High' itu lebih besar dari 'Medium', dan 'Medium' lebih besar dari 'Small'.

3. One-Hot Encoding (Untuk Kolom Setara/Nominal)

```
: # One-Hot Encoding untuk Item_Type dan Outlet_Type
# Alasan: Jenis barang dan jenis toko tidak memiliki urutan (Daging tidak Lebih tinggi dari Sayur).

df_final = pd.get_dummies(df_feature, columns=['Item_Type', 'Outlet_Type'], prefix=['Item', 'Outlet'], dtype=int)

print("One-Hot Encoding Selesai!")
# Menampilkan beberapa kolom hasil encoding
display(df_final.filter(like='Outlet_').head())
```

One-Hot Encoding Selesai!

	Outlet_Identifier	Outlet_Establishment_Year	Outlet_Size	Outlet_Location_Type	Item_Outlet_Sales	Outlet_Size_Encoded	Outlet_Location_Ty
0	OUT049	1999	Medium	Tier 1	0.856094	1	
1	OUT018	2009	Medium	Tier 3	-0.595888	1	
2	OUT049	1999	Medium	Tier 1	0.133627	1	
3	OUT010	1998	Small	Tier 3	-0.468429	0	
4	OUT013	1987	High	Tier 3	-0.352717	2	

Diterapkan pada variabel seperti Tipe Outlet. Alasannya adalah untuk menghindari 'bias urutan';



2. Label Encoding (Untuk Kolom Biner/2 Pilihan)

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

# Encoding untuk Item_Fat_Content
# Alasan: Hanya ada dua kategori utama (Low Fat & Regular), jadi cukup diubah jadi 0 dan 1
df_feature['Item_Fat_Content_Encoded'] = le.fit_transform(df_feature['Item_Fat_Content'])

print("Label Encoding Selesai!")
display(df_feature[['Item_Fat_Content', 'Item_Fat_Content_Encoded']].head())
```

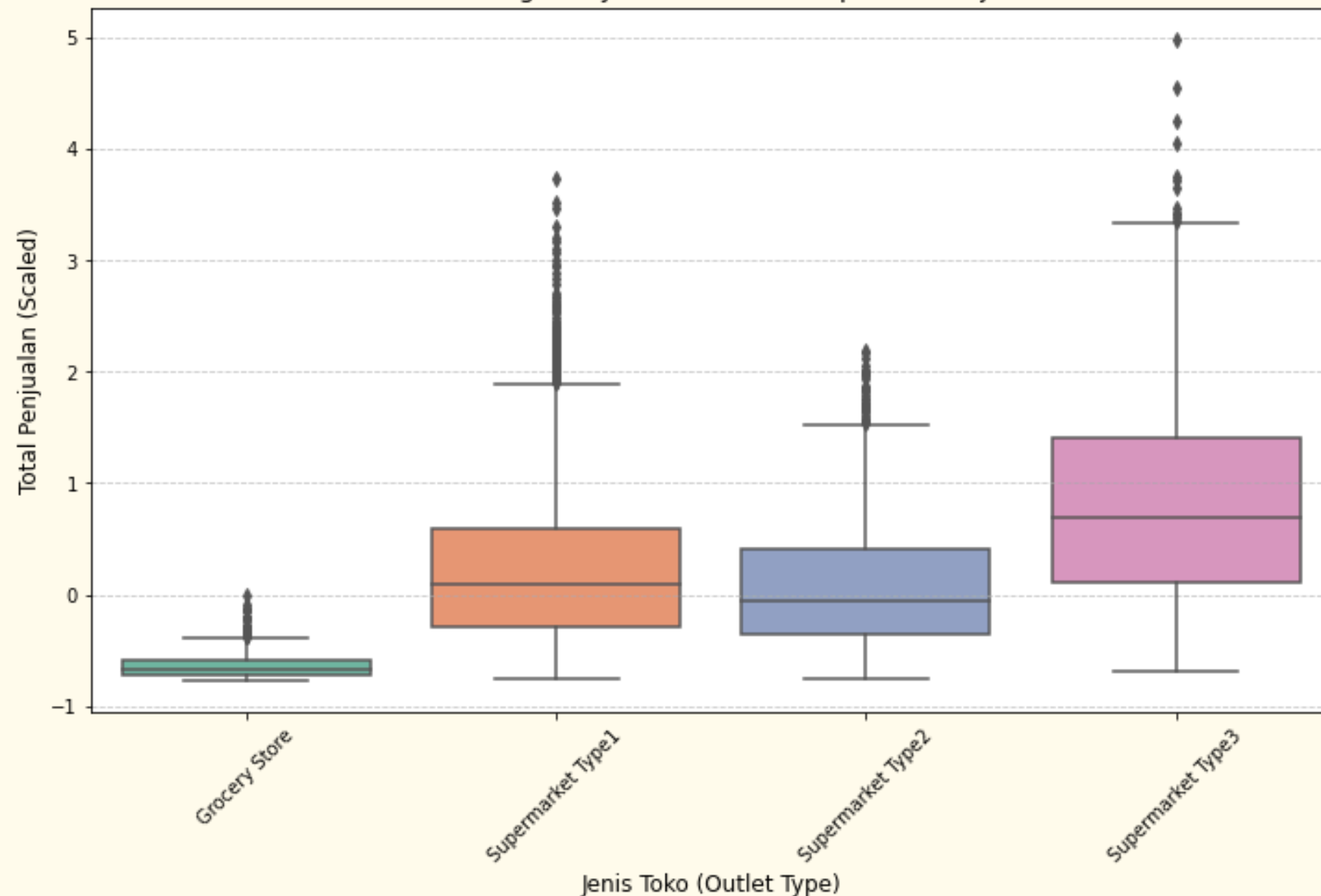
Label Encoding Selesai!

	Item_Fat_Content	Item_Fat_Content_Encoded
0	Low Fat	0
1	Regular	1
2	Low Fat	0
3	Regular	1
4	Low Fat	0

Untuk kolom seperti Item_Fat_Content yang isinya cuma "Low Fat" dan "Regular", Label Encoding paling praktis karena hanya akan menghasilkan angka 0 dan 1.

ANALISA STATISTIK

Analisis Pengaruh Jenis Toko terhadap Total Penjualan



Untuk memetakan dominasi pasar. Kita ingin tahu tipe toko mana yang paling banyak tersebar di jaringan BigMart dan tipe mana yang paling langka.

.H0: Tidak terdapat perbedaan rata-rata penjualan yang signifikan di antara berbagai jenis toko (Grocery Store, Supermarket Type 1, Type 2, dan Type 3).

H1: Terdapat perbedaan rata-rata penjualan yang signifikan di antara jenis-jenis toko tersebut.



ANALISA STATISTIK

PMF

```
pmf_data =
data_copy.groupby(['Outlet_Type',
'Outlet_Size'], observed=False).size() /
len(data_copy)
pmf_data =
pmf_data.reset_index(name='Probability')

print("=== Tabel PMF Kombinasi Jenis Toko
& Ukuran Toko ===")
display(pmf_data)
```

Insight:

- Supermarket Type 1 dengan ukuran Small merupakan kombinasi yang paling sering muncul (peluang 0.4358). Ini menunjukkan bahwa model toko tersebut adalah kontributor utama dalam jangkauan pasar BigMart.
- Tipe Grocery Store secara konsisten hanya beroperasi pada ukuran Small (0.1270). Hal ini menegaskan bahwa fokus operasional tipe ini memang ditujukan untuk skala lingkungan kecil.
- Untuk Supermarket Type 2 dan Type 3, distribusinya lebih terpusat pada ukuran Medium. Sepertinya ada standarisasi ukuran untuk tipe toko yang memiliki performa penjualan lebih tinggi agar operasionalnya lebih efisien.
- Secara keseluruhan, ekosistem BigMart didominasi ukuran Small dan Medium. Rendahnya peluang pada ukuran High menandakan strategi yang lebih mengutamakan efisiensi ruang dan jumlah outlet yang masif.



ANALISA STATISTIK

Uji statistik: Kruskal-wallis test

```
=== Hasil Uji Statistik: Kruskal-Wallis Test ===
```

```
H-statistic : 2803.3609
```

```
p-value      : 0.0000
```

```
Kesimpulan Akhir:
```

```
Tolak H0: Terdapat perbedaan distribusi penjualan yang signifikan antar jenis toko.
```

P-Value (0.0000): Karena nilainya jauh di bawah ambang batas (\$0.05), ini bukti yang sangat kuat untuk Menolak H_0 . Artinya, perbedaan angka penjualan antar jenis toko itu nyata, bukan cuma kebetulan atau karena variasi data acak.

H-statistic (2803.36): Nilai ini cukup besar, yang menunjukkan ada perbedaan yang "jauh" atau kontras antara kelompok toko yang satu dengan yang lainnya (misalnya perbedaan jomplang antara Grocery Store dan Supermarket Type 3).



OUTPUT FINAL DATASET

Data sebelum siap di analisis

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8523 entries, 0 to 8522
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Item_Identifier                       8523 non-null   object
1   Item_Weight                           7060 non-null   float64
2   Item_Fat_Content                       8523 non-null   object
3   Item_Visibility                       8523 non-null   float64
4   Item_Type                             8523 non-null   object
5   Item_MRP                             8523 non-null   float64
6   Outlet_Identifier                     8523 non-null   object
7   Outlet_Establishment_Year             8523 non-null   int64
8   Outlet_Size                           6113 non-null   object
9   Outlet_Location_Type                  8523 non-null   object
10  Outlet_Type                           8523 non-null   object
11  Item_Outlet_Sales                     8523 non-null   float64
dtypes: float64(4), int64(1), object(7)
memory usage: 799.2+ KB
```

masih terdapat missing value dan data masih berjumlah 12 kolom

df.isnull().sum()

```
Item_Identifier      0
Item_Weight          1463
Item_Fat_Content      0
Item_Visibility       0
Item_Type             0
Item_MRP              0
Outlet_Identifier     0
Outlet_Establishment_Year  0
Outlet_Size          2410
Outlet_Location_Type  0
Outlet_Type           0
Item_Outlet_Sales     0
dtype: int64
```



OUTPUT FINAL DATASET

Data sesudah siap di analisis

```
df_feature.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 8523 entries, 0 to 8522
```

```
Data columns (total 16 columns):
```

#	Column	Non-Null Count	Dtype
0	Item_Identifier	8523 non-null	object
1	Item_Weight	8523 non-null	float64
2	Item_Fat_Content	8523 non-null	category
3	Item_Visibility	8523 non-null	float64
4	Item_Type	8523 non-null	category
5	Item_MRP	8523 non-null	float64
6	Outlet_Identifier	8523 non-null	object
7	Outlet_Establishment_Year	8523 non-null	int64
8	Outlet_Size	8523 non-null	category
9	Outlet_Location_Type	8523 non-null	category
10	Outlet_Type	8523 non-null	category
11	Item_Outlet_Sales	8523 non-null	float64
12	Price_Per_Weight	8523 non-null	float64
13	Outlet_Size_Encoded	8523 non-null	category
14	Outlet_Location_Type_Encoded	8523 non-null	category
15	Item_Fat_Content_Encoded	8523 non-null	int32

```
dtypes: category(7), float64(5), int32(1), int64(1), object(2)
```

```
memory usage: 625.9+ KB
```

sudah tidak ada missing value dan kolom data bertambah menjadi 14 kolom

```
data_copy.isnull().sum()
```

Item_Identifier	0
Item_Weight	0
Item_Fat_Content	0
Item_Visibility	0
Item_Type	0
Item_MRP	0
Outlet_Identifier	0
Outlet_Establishment_Year	0
Outlet_Size	0
Outlet_Location_Type	0
Outlet_Type	0
Item_Outlet_Sales	0
dtype: int64	



INSIGHT

Jenis Toko Berpengaruh Nyata: Hasil uji statistik membuktikan bahwa perbedaan jenis toko memberikan dampak yang signifikan terhadap hasil penjualan. Perbedaan angka di grafik bukan cuma kebetulan, tapi memang ada pola yang nyata.

Supermarket Type 3 adalah tipe toko dengan rata-rata penjualan paling tinggi, sedangkan Supermarket Type 1 (Small) adalah tipe yang paling banyak jumlah cabangnya di lapangan.

Saran Bisnis: Fokus pada toko ukuran Medium sangat disarankan, karena tipe ini terbukti memiliki performa penjualan yang lebih stabil dan tinggi dibandingkan tipe lainnya.



LINK CANVA

Link



Thank You